Sesión 12 — Laboratorio

Programación Declarativa
3º del Grado en Ingeniería Informática
Funciones de orden superior

Nota

Abre GHCi y carga tus definiciones con `:load fichero.hs` (o `:l fichero.hs`). Recuerda que puedes recargar tras editar con `:reload` (o `:r`). Comprueba los tipos de tus funciones con `:type` (`:t`) antes de probarlas.
Los ejercicios marcados con ★ son los recomendados para completar durante la sesión si el tiempo es limitado.

Parte A — Reimplementando funciones de orden superior

Ejercicio 1.

Define `miMap :: (a -> b) -> [a] -> [b]` por recursión explícita (sin usar `foldr` ni `map` del preludio).

```
ghci> miMap (*2) [1,2,3]
[2,4,6]
```

Ejercicio 2. ★

Define `miFilter :: (a -> Bool) -> [a] -> [a]` por recursión explícita.

```
ghci> miFilter even [1,2,3,4,5,6]
[2,4,6]
```

Ejercicio 3.

Define `miZipWith :: (a -> b -> c) -> [a] -> [b] -> [c]` por recursión sobre dos listas a la vez.

```
ghci> miZipWith (+) [1,2,3] [10,20,30]
[11,22,33]
```

Ejercicio 4.

Define `miAny, miAll :: (a -> Bool) -> [a] -> Bool` por recursión explícita (sin usar `any`/`all`/`or`/`and` del preludio).

```
ghci> miAny even [1,3,5,6]
True
ghci> miAll even [2,4,5,6]
False
```

Parte B — Las mismas funciones, con `foldr`

Ejercicio 5.

Reescribe `miMap` (del bloque anterior) usando `foldr` en lugar de recursión explícita.

```
ghci> miMapFoldr (*2) [1,2,3]
[2,4,6]
```

Ejercicio 6. ★

Reescribe `miFilter` usando `foldr`.

```
ghci> miFilterFoldr even [1,2,3,4,5,6]
[2,4,6]
```

Ejercicio 7.

Define `miConcatMap :: (a -> [b]) -> [a] -> [b]` usando `foldr`: aplica `f` a cada elemento (obteniendo una lista por cada uno) y concatena todos los resultados.

```
ghci> miConcatMap (\x -> [x,x]) [1,2,3]
[1,1,2,2,3,3]
```

Parte C — Composición y estilo *point-free*

Ejercicio 8. ★

Define `triplicaPositivos :: [Int] -> [Int]` que, a partir de una lista, devuelva el triple de los elementos positivos, combinando `map` y `filter` mediante composición, en estilo *point-free* (sin mencionar el argumento de la lista).

```
ghci> triplicaPositivos [3,-1,4,-5,2]
[9,12,6]
```

Ejercicio 9.

Define `contarQueCumplen :: (a -> Bool) -> [a] -> Int` usando composición de `length` y `filter`, en estilo *point-free*.

```
ghci> contarQueCumplen even [1,2,3,4,5,6]
3
```

Ejercicio 10.

Define `tuberia :: [a -> a] -> a -> a` que aplique una lista de funciones, en orden, a un valor inicial (la primera función de la lista se aplica primero). Resuélvelo con `foldl`.

```
ghci> tuberia [(+1), (*2), negate] 3
-8
```

Pista: con `[(+1), (*2), negate]` y entrada `3`, el resultado debe ser `negate ((+1) 3 * 2)`, es decir, se aplican en el **mismo orden** en que aparecen en la lista.

Parte D — Funciones que devuelven funciones

Ejercicio 11.

Define `mayorPor :: Ord b => (a -> b) -> a -> a -> Bool` que, a partir de una función "clave", indique si el primer valor es mayor que el segundo, comparando sus claves.

```
ghci> mayorPor length "hola" "hi"
True
ghci> mayorPor negate 3 5
True
```

Ejercicio 12. ★

Define `validador :: [a -> Bool] -> a -> Bool` que combine una lista de predicados en uno solo, que exige que el valor cumpla **todos** los predicados de la lista.

```
ghci> validador [even, (>0)] 4
True
ghci> validador [even, (>0)] (-4)
False
```

Parte E — Combinando varias piezas

Ejercicio 13. ★

Define `procesarNotas :: [Int] -> (Double, Int, Int)` que, a partir de una lista de notas (de 0 a 10), devuelva una tupla con la media (un `Double`), el número de aprobados (nota ≥ 5) y el número de suspensos.

```
ghci> procesarNotas [3,7,5,9,2,6]
(5.333333333333333,4,2)
```

Ejercicio 14.

Define `palabrasCapitalizadas :: String -> [String]` que separe un texto en palabras y devuelva cada una con su primera letra en mayúscula (el resto de la palabra sin modificar), combinando `words`, `map` y una función auxiliar de capitalización.

```
ghci> palabrasCapitalizadas "el gato come pescado"
["El","Gato","Come","Pescado"]
```

Parte F — Ejercicio integrador: validador de formularios

Para este ejercicio es útil el predicado `lookup` del preludio, que busca un valor por clave en una lista de pares:

```
lookup :: Eq a => a -> [(a,b)] -> Maybe b
```

```
ghci> lookup "b" [("a",1), ("b",2), ("c",3)]
Just 2
ghci> lookup "z" [("a",1), ("b",2), ("c",3)]
Nothing
```

Ejercicio 15. ★

Implementa un mini-validador de formularios. Dada una lista de **reglas** `[(String, String -> Bool)]` (nombre de campo, función de validación) y un **registro** `[(String, String)]` (nombre de campo, valor introducido), define `camposInvalidos :: [(String, String -> Bool)] -> [(String, String)] -> [String]` que devuelva los nombres de los campos cuyo valor **no** supera su validación. Si un campo de las reglas no aparece en el registro, considera que tampoco es válido.

```
ghci> reglas = [("edad", \v -> length v > 0 && all (`elem` ['0'..'9']) v
    ),
                ("nombre", \v -> length v > 0)]
ghci> registro = [("edad", "abc"), ("nombre", "Ana")]
ghci> camposInvalidos reglas registro
["edad"]
```